

Fullstack Developer — Technical Assignment

Parking Violation Portal

Tan Digital

2026

Overview

Thank you for taking the time to work on this assignment. The exercise is designed to give us a realistic look at how you reason about system design, scope, and trade-offs.

You are building a **Parking Violation Portal**. The shape of the system is up to you, but it must satisfy the prerequisites and the goal slice below. Anything not explicitly specified is a decision for you to make and document.

Prerequisites

The submission must satisfy all of the following:

- **Stack:** Backend must be Go. Frontend must be React (Next.js or plain React, with TypeScript preferred).
- **Backend is modular as services.** You decide the boundaries. Containerised or not, with or without orchestration (Kubernetes, Docker Compose, etc.).
- **An API Gateway is the single endpoint** between the frontend and the backend.
- **Use the supporting infrastructure that fits the problem.** PostgreSQL, RabbitMQ (or another broker), Redis, object storage, etc. — pick what you actually need and justify it. Do not include infrastructure you are not going to use.
- **A single web app** serves both Officer and Member roles, with the right view and permissions per role.

Goal — Working Slice

Build a working slice of the portal that supports the following five flows end-to-end:

1. **An officer submits a violation** — license plate, violation type, location, timestamp, photo.
2. **The system calculates the fine** using the currently active fine-rules formula.
3. **An officer can update the fine rules.** A rule change must not change the fine on past violations.
4. **A member pays the fine** from their account.
5. **A transaction history view** shows each violation, the fine, and **which rule version was applied at the time it was issued**.

Initial Fine Formula

This is the active ruleset on day one. Your system must allow officers to publish new rule versions that change any of these values, **without affecting fines that have already been issued**.

$\text{fine} = \text{base_amount}(\text{violation_type}) \times \text{time_multiplier} \times \text{repeat_multiplier}$

base_amount — by violation type (IDR):

Violation type	Amount
expired_meter	50,000
no_parking_zone	150,000
blocking_hydrant	250,000
disabled_spot	500,000

time_multiplier — based on violation timestamp:

Time window	Multiplier
06:00 – 22:00	1.0
22:00 – 06:00	1.5

repeat_multiplier — based on prior unpaid violations on the same plate in the last 90

days:

Prior unpaid violations	Multiplier
0	1.0
1	1.5
2 or more	2.0

All numbers above are configurable via the rule version.

Payment Service (Mocked)

Mock response from payment provider in internal service.

Signature:

`PaymentService.charge(invoice_id, amount, scenario) → { status, transaction_id }`

`scenario` is a test-only argument you pass to simulate the outcome:

- "success" → returns { status: "paid", transaction_id }
- "failed" → returns { status: "failed", transaction_id }

When a member initiates payment in the web app, expose a way to choose the scenario (a dropdown or radio is fine). This is purely to let us exercise both paths.

Deliverables

Submit the following in a single Git repository:

DESIGN.md — a short design document covering:

- **Data flow diagram between services** — how a violation moves from inspector submission, through fine calculation, invoice creation, notification, and payment. Show which calls are synchronous, which are asynchronous, and which cross module boundaries.
- **ERD** — entities, relationships, and key fields. Make rule versioning and the calculation snapshot on a transaction explicit in the diagram.
- Both designs should be drawn with draw.io
- Attach designs as images

Working code — a running slice of the portal covering the five flows in *Goal*. The reviewer must be able to start it locally by following your README.

README.md — how to run it locally, plus anything you want us to know: assumptions, trade-offs, what you would do with more time.

Closing

Anything not explicitly specified in this document is up to you. Make the call you think is reasonable, document it briefly in the README. If something is genuinely ambiguous and the answer would materially change your design, please reach out before assuming; otherwise, document your assumption and proceed.

For any clarifying questions, reply to the email this assignment came from.

Good luck — we look forward to seeing what you build.